«Разработка класса. Использование реализованного в виде модуля класса в приложении»

Дисциплина: Основы алгоритмизации и программирования
Специальность: 09.02.07 «Информационные системы и программирование»

Цель занятия

Освоить принципы объектно-ориентированного программирования: создание класса, описание его полей и методов, использование конструкторов и деструкторов. Научиться оформлять класс в виде отдельного модуля (заголовочного файла `.h` и файла реализации `.cpp`) и использовать его в приложении.

Задачи занятия

Знать:

- понятие класса, объекта, поля, метода;
- модификаторы доступа `public`, `private`, `protected`;
- назначение конструкторов и деструкторов;
- принцип разделения интерфейса и реализации (модульность);
- структуру заголовочного файла и файла реализации.

Уметь:

- создавать класс с полями и методами;
- использовать конструкторы для инициализации объектов;
- реализовывать методы класса;
- оформлять класс в виде модуля (`.h` + `.cpp`);
- подключать модуль в приложении и использовать объекты класса.

Теоретическая часть

1. Зачем нужны классы

Представьте, что вы пишете программу для библиотеки. У вас есть книги, читатели, выдачи. Если хранить всё в отдельных переменных и массивах, код быстро превратится в кашу: где-то имя читателя, где-то его номер, где-то список книг. А если добавить новую сущность - например, журнал - придётся переписывать половину программы.

Классы решают эту проблему. Класс - это шаблон, по которому создаются объекты. Класс описывает, какие поля (данные) и методы (функции) будут у объекта. Объект - это конкретный экземпляр класса. Например, класс `Book` описывает книгу вообще, а объект `book1` - конкретную книгу с названием «Война и мир» и автором «Л. Н. Толстой».

Главная идея классов - инкапсуляция. Это значит, что данные класса скрыты от внешнего мира, а доступ к ним осуществляется через методы. Это защищает данные от случайного изменения и упрощает поддержку кода. Например, если у книги есть поле `year` (год издания), мы не хотим, чтобы кто-то извне записал туда `-500`. Поэтому мы делаем поле приватным, а доступ к нему - только через метод `setYear`, который проверяет корректность значения.

2. Как объявить класс в C++

Класс объявляется с помощью ключевого слова `class`. Внутри класса размещаются поля и методы. Для скрытия данных используются модификаторы доступа: `private` (доступ только внутри класса), `public` (доступ извне), `protected` (доступ внутри класса и наследников). По умолчанию в `class` все члены приватные, в `struct` - публичные.

Рассмотрим простейший класс:

```cpp
class Book {

private:

    string title;      // название книги
```

```
    string author;      // автор
    int year;           // год издания
    int pages;          // количество страниц

public:
    void print() {
        cout << title << ", " << author << endl;
    }
};
```

Здесь `title`, `author`, `year`, `pages` - поля класса. `print()` - метод. Поля скрыты (`private`), а метод `print()` доступен извне (`public`). Это и есть инкапсуляция.

3. Создание объекта

Объект класса создаётся как обычная переменная:

```
cpp

Book book1;   // создаём объект book1
```

После создания объект содержит поля с неопределёнными значениями (так называемый «мусор»). Чтобы задать начальные значения, используются конструкторы.

4. Конструкторы

Конструктор - это специальный метод, который вызывается автоматически при создании объекта. Его имя совпадает с именем класса. Конструктор не возвращает значение. Конструктор может быть с параметрами и без параметров.

Конструктор без параметров (конструктор по умолчанию) задаёт значения полей по умолчанию:

```cpp
class Book {
private:
    string title;
    string author;
    int year;
    int pages;

public:
    Book() {
        title = "Без названия";
        author = "Неизвестен";
        year = 0;
        pages = 0;
    }
};
```

Конструктор с параметрами позволяет задать значения полей при создании объекта:

```cpp
class Book {
private:
    string title;
```

```
    string author;
    int year;
    int pages;
public:
    Book(string t, string a, int y, int p) {
        title = t;
        author = a;
        year = y;
        pages = p;
    }
};
```

Теперь объект можно создать так:

```
cpp
Book book1("Война и мир", "Л. Н. Толстой", 1869, 1300);
```

При создании объекта `book1` автоматически вызывается конструктор с параметрами, и поля получают нужные значения.

Можно также использовать список инициализации - более короткую и эффективную запись конструктора:

```
cpp
Book(string t, string a, int y, int p)
: title(t), author(a), year(y), pages(p) {}
```

Эта запись означает то же самое: поля `title`, `author`, `year`, `pages` инициализируются значениями параметров `t`, `a`, `y`, `p`.

5. Деструкторы

Деструктор - это метод, который вызывается автоматически при уничтожении объекта. Его имя начинается с тильды `~` и совпадает с именем класса. Деструктор используется для освобождения ресурсов (например, памяти, выделенной в конструкторе).

```cpp
class Book {

public:

    ~Book() {

        cout << "Книга \"" << title << "\" удалена" << endl;

    }

};
```

Деструктор вызывается, когда объект выходит из области видимости (например, при завершении программы или при выходе из блока `{ ... }`).

6. Геттеры и сеттеры

Геттер - это метод, который возвращает значение поля. Сеттер - это метод, который изменяет значение поля. Геттеры и сеттеры позволяют контролировать доступ к приватным полям.

```cpp
class Book {

private:

    string title;

    int year;



public:
```

```
    string getTitle() { return title; }

    int getYear() { return year; }



    void setTitle(string t) {

        if (t != "") title = t;

    }



    void setYear(int y) {

        if (y > 0 && y < 2100) year = y;

    }

};
```

В сеттере можно проверять корректность значения. Например, год издания не может быть отрицательным или больше 2100.

7. Методы класса

Метод - это функция, которая принадлежит классу и может работать с его полями. Методы объявляются внутри класса. Если метод определён внутри класса, он автоматически считается встроенным (`inline`). Если метод определён вне класса, используется оператор `::` (разрешение области видимости):

```
cpp

class Book {

public:

    void print();

};
```

```
void Book::print() {

    cout << title << ", " << author << endl;

}
```

Метод `print()` объявлен внутри класса, но определён вне его. Это удобно, когда метод большой.

8. Модульность: заголовочный файл и файл реализации

Когда программа разрастается, держать весь код в одном файле неудобно. Класс можно оформить в виде модуля - двух файлов: заголовочного `.h` и файла реализации `.cpp`.

Заголовочный файл содержит объявление класса: поля, прототипы методов, конструкторы. Он подключается в основной программе с помощью `#include "Book.h"`. Файл реализации содержит определения методов.

Пример заголовочного файла `Book.h`:

```
cpp

#ifndef BOOK_H      // защита от повторного включения

#define BOOK_H



#include <string>

using namespace std;



class Book {

private:

    string title;
```

```
    string author;
    int year;
    int pages;

public:
    Book();
    Book(string t, string a, int y, int p);
    ~Book();

    void setTitle(string t);
    void setAuthor(string a);
    void setYear(int y);
    void setPages(int p);

    string getTitle();
    string getAuthor();
    int getYear();
    int getPages();

    bool isThick();
    void print();
};
```

```
#endif
```

Конструкции `#ifndef`, `#define`, `#endif` называются include guard (защита от повторного включения). Они предотвращают повторное включение заголовочного файла, если он подключается несколько раз. Это важно, потому что повторное объявление класса вызовет ошибку компиляции.

Пример файла реализации `Book.cpp`:

cpp

```cpp
#include "Book.h"

#include <iostream>

using namespace std;



Book::Book() {

    title = "Без названия";

    author = "Неизвестен";

    year = 0;

    pages = 0;

}



Book::Book(string t, string a, int y, int p) {

    title = t;

    author = a;

    year = y;

    pages = p;

}
```

```
Book::~Book() {
    cout << "Книга \"" << title << "\" удалена" << endl;
}

void Book::setTitle(string t) { title = t; }
void Book::setAuthor(string a) { author = a; }
void Book::setYear(int y) { year = y; }
void Book::setPages(int p) { pages = p; }

string Book::getTitle() { return title; }
string Book::getAuthor() { return author; }
int Book::getYear() { return year; }
int Book::getPages() { return pages; }

bool Book::isThick() {
    return pages > 500;
}

void Book::print() {
    cout << "Книга: " << title << ", автор: " << author
         << ", год: " << year << ", страниц: " << pages << endl;
```

```
}
```

Обратите внимание: перед именем метода указывается имя класса и оператор `::`, например `void Book::print()`. Это говорит компилятору, что метод `print` принадлежит классу `Book`.

9. Использование класса в приложении

Основная программа подключает заголовочный файл и создаёт объекты класса:

```cpp
#include <iostream>

#include "Book.h"

using namespace std;



int main() {

    Book book1("Война и мир", "Л. Н. Толстой", 1869, 1300);

    book1.print();



    Book book2;

    book2.setTitle("Преступление и наказание");

    book2.setAuthor("Ф. М. Достоевский");

    book2.setYear(1866);

    book2.setPages(671);

    book2.print();
```

```
    return 0;

}
```

При компиляции проекта из нескольких файлов важно, чтобы все `.cpp`-файлы были добавлены в проект. В среде разработки (Visual Studio, Code::Blocks, CLion) это делается через меню «Добавить файл в проект». При компиляции из командной строки нужно указать все `.cpp`-файлы: `g++ main.cpp Book.cpp -o program`.

10. Статические поля и методы

Статическое поле принадлежит классу, а не объекту. Оно объявляется с ключевым словом `static` и инициализируется вне класса. Все объекты класса разделяют одно и то же статическое поле.

Статический метод - это метод, который не привязан к конкретному объекту и вызывается через имя класса. Он не имеет доступа к нестатическим полям, но может обращаться к статическим.

Пример подсчёта количества созданных объектов:

```cpp
class Book {

private:

    static int count;



public:

    Book() { count++; }

    ~Book() { count--; }

    static int getCount() { return count; }

};

int Book::count = 0;
```

Теперь можно узнать количество активных объектов:

cpp

```
cout << "Активных книг: " << Book::getCount() << endl;
```

11. Перегрузка операторов

Перегрузка операторов позволяет использовать объекты класса с привычными операторами (`+`, `==`, `<<` и др.). Например, перегрузка `<<` для вывода:

```cpp
ostream& operator<<(ostream& os, const Book& b) {
    os << b.getTitle() << ", " << b.getAuthor();
    return os;
}
```

Теперь можно выводить объект так же, как обычную переменную:

cpp

```
cout << book1 << endl;
```

12. Преимущества модульного подхода

Разделение интерфейса и реализации. Пользователю класса не нужно знать, как устроены методы, - достаточно заголовочного файла.

Повторное использование. Один и тот же модуль можно подключать в разные проекты.

Упрощение сопровождения. Изменения в реализации не затрагивают код, который использует класс.

Ускорение компиляции. При изменении `.cpp`-файла перекомпилируется только он, а не весь проект.

5. Практическая часть

Задача I. Разработка класса

Условие. Разработать класс в соответствии с вариантом задания. Класс должен содержать:

- не менее трёх полей разных типов;
- конструктор с параметрами;
- конструктор без параметров (или конструктор по умолчанию);
- деструктор;
- методы для доступа к полям (геттеры и сеттеры);
- метод для вывода информации об объекте;
- не менее одного метода, выполняющего вычисления или обработку данных.

Теоретическая сводка-подсказка. Класс объявляется с помощью ключевого слова `class`. Поля и методы размещаются внутри фигурных скобок. Для скрытия данных используйте модификатор `private`, для доступа извне - `public`. Конструктор имеет то же имя, что и класс, и не возвращает значение. Деструктор начинается с `~`. Геттеры возвращают значение поля, сеттеры изменяют его с проверкой корректности. Метод `print()` выводит информацию об объекте на экран.

Варианты заданий:

1. Класс `Book` (книга): название, автор, год издания, количество страниц. Метод - определение «толстая ли книга» (больше 500 страниц).
2. Класс `Student` (студент): фамилия, имя, курс, средний балл. Метод - определение, является ли студент отличником (средний балл ≥ 4.5).
3. Класс `Car` (автомобиль): марка, модель, год выпуска, объём двигателя. Метод - расчёт налога в зависимости от объёма двигателя.
4. Класс `Point` (точка на плоскости): координаты `x`, `y`. Методы - расстояние до начала координат, расстояние до другой точки.
5. Класс `Rectangle` (прямоугольник): ширина, высота. Методы - площадь, периметр.
6. Класс `Circle` (круг): радиус. Методы - площадь, длина окружности.
7. Класс `BankAccount` (банковский счёт): номер счёта, баланс, владелец. Методы - пополнение, снятие, проверка баланса.
8. Класс `Product` (товар): название, цена, количество на складе. Метод - расчёт общей стоимости.

9. Класс `Date` (дата): день, месяц, год. Методы - проверка корректности даты, вывод в формате «ДД.ММ.ГГГГ».
10. Класс `Time` (время): часы, минуты, секунды. Методы - перевод в секунды, вывод в формате «ЧЧ:ММ:СС».
11. Класс `Complex` (комплексное число): действительная и мнимая части. Методы - сложение, вычитание, умножение.
12. Класс `Fraction` (дробь): числитель, знаменатель. Методы - сложение, вычитание, сокращение дроби.
13. Класс `Employee` (сотрудник): фамилия, должность, оклад, стаж. Метод - расчёт премии в зависимости от стажа.
14. Класс `Triangle` (треугольник): длины трёх сторон. Методы - проверка существования, площадь по формуле Герона.
15. Класс `Matrix2x2` (матрица 2×2): четыре элемента. Методы - определитель, умножение на число.
16. Класс `StringWrapper` (обёртка над строкой): строка. Методы - длина, перевод в верхний регистр, проверка на палиндром.
17. Класс `Vector3D` (трёхмерный вектор): координаты `x`, `y`, `z`. Методы - длина, скалярное произведение.
18. Класс `Temperature` (температура): значение в градусах Цельсия. Методы - перевод в Фаренгейты, в Кельвины.
19. Класс `Phone` (телефон): модель, номер, баланс. Методы - звонок (уменьшение баланса), пополнение.

Пример решения (вариант 1 - класс `Book`):

```
cpp
```

```cpp
// Book.h

#ifndef BOOK_H

#define BOOK_H



#include <string>

using namespace std;



class Book {
```

```
private:
    string title;
    string author;
    int year;
    int pages;

public:
    Book();
    Book(string t, string a, int y, int p);
    ~Book();

    void setTitle(string t);
    void setAuthor(string a);
    void setYear(int y);
    void setPages(int p);

    string getTitle();
    string getAuthor();
    int getYear();
    int getPages();

    bool isThick();
```

```
    void print();
};
#endif
```

cpp

```
// Book.cpp
#include "Book.h"
#include <iostream>
using namespace std;

Book::Book() {
    title = "Без названия";
    author = "Неизвестен";
    year = 0;
    pages = 0;
}

Book::Book(string t, string a, int y, int p) {
    title = t;
    author = a;
    year = y;
    pages = p;
}
```

```
Book::~Book() {
    cout << "Книга \"" << title << "\" удалена" << endl;
}

void Book::setTitle(string t) { title = t; }
void Book::setAuthor(string a) { author = a; }
void Book::setYear(int y) { year = y; }
void Book::setPages(int p) { pages = p; }

string Book::getTitle() { return title; }
string Book::getAuthor() { return author; }
int Book::getYear() { return year; }
int Book::getPages() { return pages; }

bool Book::isThick() {
    return pages > 500;
}

void Book::print() {
    cout << "Книга: " << title << ", автор: " << author
         << ", год: " << year << ", страниц: " << pages << endl;
```

```
}
```

cpp

```cpp
// main.cpp

#include <iostream>

#include "Book.h"

using namespace std;



int main() {

    Book book1("Война и мир", "Л. Н. Толстой", 1869, 1300);

    book1.print();

    cout << "Толстая ли книга? " << (book1.isThick() ? "Да" : "Нет") <<
endl;



    Book book2;

    book2.print();



    return 0;

}
```

Критерии оценивания задачи I (максимум 6 баллов):

- класс объявлен с полями и методами - 1 балл;
- использованы модификаторы доступа `private` и `public` - 1 балл;
- реализованы конструкторы и деструктор - 1 балл;
- реализованы геттеры и сеттеры - 1 балл;
- реализован метод вычисления или обработки - 1 балл;
- программа компилируется и работает корректно - 1 балл.

Задача II. Оформление класса в виде модуля

Условие. Оформить класс из задачи I в виде модуля: заголовочный файл `.h` и файл реализации `.cpp`. Использовать include guard. Подключить модуль в основной программе.

Теоретическая сводка-подсказка. Заголовочный файл содержит объявление класса, прототипы методов и конструкторов. Файл реализации содержит определения методов. Include guard защищает от повторного включения:

```
cpp
```

```cpp
#ifndef BOOK_H

#define BOOK_H

// объявление класса

#endif
```

Для подключения модуля в основной программе используйте `#include "Book.h"`. Убедитесь, что `.cpp`-файл модуля добавлен в проект.

Критерии оценивания задачи II (максимум 4 балла):

- создан заголовочный файл `.h` с объявлением класса - 1 балл;
- создан файл реализации `.cpp` с определениями методов - 1 балл;
- использован include guard - 1 балл;
- модуль успешно подключён и работает - 1 балл.

Задача III. Использование модуля в приложении

Условие. Написать приложение, которое использует модуль из задачи II. Приложение должно:

- создавать несколько объектов класса (не менее трёх);
- демонстрировать работу конструкторов с параметрами и без параметров;
- вызывать методы класса (геттеры, сеттеры, методы обработки);
- выводить информацию об объектах;
- демонстрировать работу деструкторов (вывести сообщения при удалении объектов).

Теоретическая сводка-подсказка. Объекты класса создаются как обычные переменные: `Book book1(...);`. При создании объекта вызывается конструктор. При выходе из области видимости вызывается деструктор. Чтобы явно увидеть работу деструктора, можно создать объект в блоке `{ ... }` или использовать динамическое выделение памяти через `new`/`delete`.

Пример:

```cpp
#include <iostream>

#include "Book.h"

using namespace std;



int main() {

    Book book1("Война и мир", "Л. Н. Толстой", 1869, 1300);

    book1.print();



    Book book2;

    book2.setTitle("Преступление и наказание");

    book2.setAuthor("Ф. М. Достоевский");

    book2.setYear(1866);

    book2.setPages(671);

    book2.print();



    {

        Book book3("Мастер и Маргарита", "М. А. Булгаков", 1967, 480);
```

```
        book3.print();

    }  // здесь вызовется деструктор book3


    return 0;

}  // здесь вызовутся деструкторы book1 и book2
```

Критерии оценивания задачи III (максимум 4 балла):

- создано не менее трёх объектов - 1 балл;
- продемонстрированы конструкторы с параметрами и без - 1 балл;
- вызваны методы класса и выведена информация - 1 балл;
- продемонстрирована работа деструкторов - 1 балл.

Задача IV. Расширение класса

Условие. Добавить в класс из задачи I новые возможности:

- статическое поле для подсчёта количества созданных объектов;
- статический метод для получения этого значения;
- перегрузку оператора (например, `+`, `==`, `<<`) для класса;
- метод, который возвращает строковое представление объекта.

Теоретическая сводка-подсказка. Статическое поле принадлежит классу, а не объекту. Оно объявляется с ключевым словом `static` и инициализируется вне класса:

```
cpp

class Book {

private:

    static int count;

public:

    Book(...) { count++; }

    ~Book() { count--; }
```

```
    static int getCount() { return count; }

};

int Book::count = 0;
```

Перегрузка операторов позволяет использовать объекты класса с привычными операторами. Например, перегрузка `<<` для вывода:

```
cpp

ostream& operator<<(ostream& os, const Book& b) {

    os << b.title << ", " << b.author;

    return os;

}
```

Критерии оценивания задачи IV (максимум 4 балла):

- добавлено статическое поле и статический метод - 1 балл;
- реализована перегрузка оператора - 1 балл;
- реализован метод строкового представления - 1 балл;
- программа компилируется и работает корректно - 1 балл.

6. Общая оценка за занятие

Максимум - 18 баллов.

Оценка «отлично» ставится при 16–18 баллах, «хорошо» - при 12–15 баллах, «удовлетворительно» - при 8–11 баллах, «неудовлетворительно» - при меньшем количестве баллов.

7. Требования к отчёту

Отчёт должен содержать:

- формулировку индивидуального задания;
- описание структуры класса (поля, методы, модификаторы доступа);
- исходный код файла реализации `.cpp`;
- исходный код основной программы;
- описание работы конструкторов и деструкторов;

- результаты работы программы (скриншоты);
- выводы о преимуществах модульного подхода.